

111010547 資工四 莊偉真

Mid-term Project: Machine Learning Algorithm

Introduction: What is Machine Learning?

Machine Learning (ML) is a branch of artificial intelligence that enables computers to learn from data and make predictions or decisions without being explicitly programmed for every scenario. Instead of following rigid rules, ML algorithms identify patterns in data and improve their performance over time.

Real-World Example: Think about how Netflix recommends movies you might like. It analyzes your viewing history, compares it with millions of other users, and predicts what you'll enjoy watching next. This is machine learning in action!

The Algorithm: K-Nearest Neighbors (KNN)

For this project, I'll focus on the **K-Nearest Neighbors (KNN)** algorithm, one of the simplest yet most effective classification algorithms.

How KNN Works:

1. Imagine you have a dataset of items with known categories (like fruits classified by color and size)
2. When a new, unknown item appears, KNN looks at the K closest items to it in the dataset
3. It then assigns the new item to the category that most of its neighbors belong to
4. The "K" represents how many neighbors to consider (typically an odd number like 3, 5, or 7)

Simple Analogy: Imagine you move to a new neighborhood and want to know the best coffee shop. You ask your 5 nearest neighbors. If 4 of them recommend "Coffee Haven" and 1 recommends "Bean There," you'd probably choose Coffee Haven. That's exactly how KNN works!

Dataset: Iris Flower Classification

I'll use the famous **Iris dataset**, which contains measurements of 150 iris flowers from three different species:

- Setosa
- Versicolor
- Virginica

Each flower has four features measured:

- Sepal length (cm)
- Sepal width (cm)
- Petal length (cm)
- Petal width (cm)

Our goal is to train a model that can predict which species a flower belongs to based on these measurements.

Implementation Code

```
# Import necessary libraries
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load the Iris dataset
iris = load_iris()
X = iris.data # Features
y = iris.target # Target labels

# Split data into training (70%) and testing (30%) sets
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create and train the KNN model with k=5
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# Make predictions on test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Testing different K values to find optimal
k_values = range(1, 21)
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

```

```

pred = knn.predict(X_test)
accuracies.append(accuracy_score(y_test, pred))

# Plot K vs Accuracy
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracies, marker='o')
plt.xlabel('K Value')
plt.ylabel('Accuracy')
plt.title('KNN: K Value vs Accuracy')
plt.grid(True)
plt.show()

print(f"\nBest K value: {k_values[np.argmax(accuracies)]} with accuracy: {max(accuracies)*100:.2f}%")

```

Results Analysis

Expected Accuracy: The KNN algorithm typically achieves 95-98% accuracy on the Iris dataset with optimal K values.

Key Findings:

- **Optimal K Value:** Through testing, K=5 or K=7 usually provides the best balance between underfitting and overfitting
- **Confusion Matrix:** Shows that most misclassifications occur between Versicolor and Virginica species, as they have similar measurements
- **Feature Importance:** Petal length and petal width are the most distinguishing features for classification

Algorithm Advantages

- Simple to understand and implement
- No training phase required (lazy learning)
- Works well with small to medium datasets
- Effective for multi-class classification problems

- No assumptions about data distribution

Algorithm Limitations

- Computationally expensive for large datasets (must calculate distance to all points)
- Sensitive to irrelevant features and feature scaling
- Requires careful selection of K value
- Struggles with high-dimensional data (curse of dimensionality)
- Memory-intensive as it stores all training data

Potential Improvements

1. **Feature Scaling:** Normalize features to prevent bias toward features with larger ranges
2. **Distance Metrics:** Experiment with different distance calculations (Euclidean, Manhattan, Minkowski)
3. **Weighted KNN:** Give more weight to closer neighbors
4. **Cross-Validation:** Use k-fold cross-validation to better estimate model performance
5. **Feature Engineering:** Create new features or remove irrelevant ones to improve accuracy

Common Questions & Answers

▼ Q: Why choose KNN over other algorithms?

A: KNN is intuitive and easy to explain to non-technical audiences. It doesn't require complex mathematical understanding, making it perfect for demonstrating machine learning concepts. Additionally, it performs well on the Iris dataset, which is ideal for educational purposes.

▼ Q: What happens if you choose a bad K value?

A: If K is too small (like $K=1$), the model becomes sensitive to noise and outliers, leading to overfitting. If K is too large, the model becomes too general and may underfit the data. That's why testing different K values is crucial.

▼ **Q: How does KNN handle new data it hasn't seen before?**

A: KNN doesn't "learn" in the traditional sense. When given new data, it calculates the distance to all training examples and finds the K nearest ones. This makes it a "lazy learner" - it doesn't build a model during training, but instead stores all training data for use during prediction.

▼ **Q: Can this be applied to real-world problems?**

A: Absolutely! KNN is used in recommendation systems, pattern recognition, medical diagnosis, credit rating, and image recognition. For example, hospitals use similar algorithms to predict disease based on patient symptoms and medical history.

Attribution Statement

This project represents my original work with the following acknowledgments:

- The K-Nearest Neighbors implementation uses the scikit-learn library, documented at <https://scikit-learn.org/>
- The Iris dataset is a standard dataset available through scikit-learn, originally collected by Ronald Fisher (1936)
- Code structure was adapted from scikit-learn documentation and standard ML tutorial